



Ciencias de la Computación

Compiladores (CS3025)

Teo. 1 — Lab. 11 — 2026-2

Analizador léxico y sintáctico de expresiones

Informe de desarrollo

Integrantes

Nayeli Guerrero

David Quispe

Evaluación Continua 1

Índice

1. Introducción	2
1.1. Objetivo	2
1.2. Alcance	2
1.3. Organización del informe	2
2. Análisis previo	2
2.1. Estado inicial	2
2.2. Qué exigían los casos de prueba	2
2.3. Por dónde empezar	2
3. Desarrollo	3
3.1. Definición de los tokens	3
3.2. Analizador léxico	3
3.3. Estructura del árbol	4
3.4. Analizador sintáctico	4
3.5. Programa principal	5
4. Validación	6
4.1. Ejecución del conjunto de pruebas	6
4.2. Casos de error	6
4.3. Resultados	6
5. Conclusiones	6
A. Compilación y ejecución	7

1. Introducción

1.1. Objetivo

Extender un analizador de expresiones aritméticas para que acepte los diez casos de prueba entregados con el enunciado. El programa recibe un archivo con una expresión, produce la lista de tokens que la componen y construye el árbol de sintaxis abstracta (AST) correspondiente, exportándolo en formato DOT para poder visualizarlo.

1.2. Alcance

El lenguaje reconocido admite:

- Operadores `+`, `-`, `*`, `/` y `**` (potencia), con la precedencia habitual y paréntesis para agruparlos.
- Números enteros y decimales, por ejemplo `2` y `50.5`.
- Identificadores de variables, como `x1` o `x2`.
- Las funciones `sqrt`, `abs`, `max` y `min`.

El trabajo se limita al análisis léxico y sintáctico. No se implementa la evaluación de las expresiones, ya que ninguno de los casos de prueba la requiere.

1.3. Organización del informe

La sección 2 describe el estado inicial del código y el análisis que hicimos antes de programar. La sección 3 detalla las modificaciones en el orden en que se realizaron, con el motivo de cada una. La sección 4 presenta las pruebas y la sección 5 recoge lo aprendido.

2. Análisis previo

2.1. Estado inicial

El código de partida ya resolvía buena parte del problema. El escáner reconocía los cinco operadores, los paréntesis, los números enteros, los identificadores y la palabra `sqrt`. El parser implementaba una gramática de precedencias con tres niveles y construía un AST con tres tipos de nodo: `BinaryExp`, `NumberExp` e `IdExp`, más `SqrtExp` para la raíz cuadrada.

2.2. Qué exigían los casos de prueba

En lugar de empezar a escribir código, revisamos primero los diez archivos de `inputs/` y anotamos qué aparecía en ellos que el analizador todavía no podía procesar. El resultado fue la tabla 1.

2.3. Por dónde empezar

Decidimos atacar primero la coma. Pesaron dos razones. La primera es de cobertura: ocho de las diez entradas la contienen, así que sin ella la mayoría de las pruebas seguiría fallando. La segunda es de diseño: admitir varios argumentos obliga a responder cómo se almacenan en el árbol, y esa respuesta condiciona el resto del trabajo.

Aquí resultó clave la indicación del profesor: *el argumento de una función es una lista de expresiones*. Con esa idea, un único tipo de nodo puede representar tanto una función de un argumento como una de varios, y no hace falta una clase distinta por cada función.

Elemento	Dónde aparece	Problema
Coma separando argumentos	<code>max(4,7)</code> (input2) y otras siete entradas	No existía un token para ,
Números decimales	<code>50.5 + (10.25 * 3)</code> (input3)	El escáner solo leía dígitos enteros
Funciones <code>max</code> , <code>min</code> y <code>abs</code>	inputs 2, 4, 5, 6, 7, 8, 9 y 10	Únicamente <code>sqrt</code> estaba contemplada

Cuadro 1: Diferencias entre el analizador inicial y los casos de prueba.

3. Desarrollo

Las modificaciones siguieron el orden natural de dependencia entre los módulos. Cada archivo responde una pregunta que el siguiente necesita ya resuelta.

3.1. Definición de los tokens

El token es la unidad más pequeña del análisis, por lo que se modificó primero. Si el escáner no puede nombrar una coma, el parser jamás llegará a verla.

Se añadieron dos valores al `enum Type` de `token.h`:

```
COMMA, // nuevo: la ','
FUNC,  // reemplaza a SQRT
```

Optamos por sustituir `SQRT` por un `FUNC` genérico en vez de crear `MAX`, `MIN` y `ABS`. Las cuatro funciones comparten la misma forma léxica, un nombre seguido de un paréntesis, así que el escáner no gana nada distinguiéndolas: le basta con guardar el nombre en `token.text` y dejar que el parser decida qué hacer con él. El resultado es un tipo de token en lugar de cuatro.

En `token.cpp` se agregaron los casos correspondientes a la sobrecarga del operador `<<`, responsable de generar los archivos `_tokens.txt`.

3.2. Analizador léxico

Con el vocabulario definido, el trabajo pasó a `nextToken()`.

Números decimales. Tras consumir la parte entera se comprueba si sigue un punto que tenga al menos un dígito detrás:

```
if (current + 1 < input.length() && input[current] == '.' &&
    es_digito(input[current + 1])) {
```

La segunda condición no es un detalle menor. Sin ella una entrada como `5.` se aceptaría como número y `stod` recibiría después una cadena mal formada.

Funciones. Al terminar de leer una palabra se compara el lexema contra el conjunto de funciones conocidas:

```
static bool es_funcion(const string& lexema) {
    static const set<string> funciones = {"sqrt", "max", "min", "abs"};
    return funciones.count(lexema) > 0;
}
```

Si el lexema pertenece al conjunto se emite un `FUNC`; en caso contrario se trata de un `ID`, es decir una variable como `x1`. Incorporar una función nueva al lenguaje se reduce a añadir una palabra a ese conjunto.

La coma. Se agregó un caso al `switch` de operadores:

```
case ',',': token = new Token(Token::COMMA, c); break;
```

Un error detectado durante el proceso. La condición que filtraba los operadores era `strchr("+/*()=, c)`, pero el `switch` que venía a continuación no contemplaba ni `;` ni `=`. Si la entrada incluía alguno de esos caracteres, la variable `token` se devolvía sin inicializar, lo que constituye comportamiento indefinido y puede provocar cualquier resultado en tiempo de ejecución. Eliminamos ambos caracteres de la lista para que se procesen en la rama `else` y se reporten como `ERR`, que es el tratamiento correcto dado que el lenguaje no los usa.

3.3. Estructura del árbol

Antes de tocar el parser conviene decidir qué debe construir.

La clase `SqrtExp` almacenaba un solo hijo, lo que resulta insuficiente para una llamada como `max(20, 15, 30)`. Aplicando la idea de la lista de expresiones, la generalizamos a un nodo con varios hijos:

```
class FuncExp : public Exp {
public:
    string name;
    list<Exp*> args;
    FuncExp(const string& name, const list<Exp*>& args);
    ~FuncExp();
    void toDot(std::ostream& out, int& id) const override;
};
```

Un solo nodo cubre ambas situaciones. La llamada `sqrt(x)` produce un `FuncExp` con un elemento en la lista y `max(a, b, c)` produce el mismo tipo de nodo con tres. No fue necesario escribir clases separadas para `max`, `min` y `abs`.

Su método `toDot()` recorre la lista y traza una arista por argumento, con lo que el dibujo del árbol resulta correcto sea cual sea la cantidad:

```
for (Exp* arg : args) {
    int childId = id;
    arg->toDot(out, id);
    out << " node" << myId << " -> node" << childId << ";\n";
}
```

Cambio asociado. La clase `NumberExp` guardaba el valor en un `int`. El literal `50.5` del `input3` quedaba truncado a `50`, por lo que el campo pasó a ser `double` y la conversión en el parser cambió de `stoi` a `stod`.

3.4. Analizador sintáctico

Definidos los tokens y los nodos, el parser establece la relación entre ambos. La gramática resultante es:

```
P    -> CE
CE   -> E (('+' | '-' ) E)*
E    -> T (('*' | '/' ) T)*
T    -> F ('**' T)?
F    -> NUM | ID | '(' CE ')' | FUNC '(' ARGS ')'
ARGS -> CE (',' CE)*
```

La regla ARGS. Traduce directamente la noción de lista de expresiones:

```
list<Exp*> Parser::parseArgs() {
    list<Exp*> args;
    args.push_back(parseCE());
    while (match(Token::COMMA)) {
        args.push_back(parseCE());
    }
    return args;
}
```

Cada argumento se analiza invocando `parseCE()`, la regla de mayor alcance de la gramática. Gracias a ello el `input6` funciona sin ningún tratamiento especial: en `max(10+10, 5*0)` cada argumento es una expresión completa y no simplemente un número.

Tratamiento de errores. El código original invocaba `match(Token::RPAREN)` e ignoraba el valor devuelto. Una entrada como `(2 + 3`, sin cerrar el paréntesis, se aceptaba en silencio y el árbol resultante era incorrecto. Sustituimos esas llamadas por un método que exige la presencia del token:

```
void Parser::expect(Token::Type ttype, const string& esperado) {
    if (!match(ttype))
        throw runtime_error("Error sintactico: se esperaba " + esperado +
                             " y se encontro " + describir(current));
}
```

Aridad de las funciones. La gramática por sí sola acepta cualquier número de argumentos, pero `sqrt(2, 3)` carece de sentido. Esta comprobación no es sintáctica sino semántica, y se realiza una vez construida la lista:

```
static bool aridad_valida(const string& name, size_t n) {
    if (name == "sqrt" || name == "abs") return n == 1;
    if (name == "max" || name == "min") return n >= 2;
    return false;
}
```

Asociatividad de la potencia. La regla `T -> F ('**' T)?` se invoca a sí misma por la derecha, con lo cual `2**3**2` se agrupa como `2**(3**2)`, tal como se interpreta en matemáticas. Un bucle `while`, como el empleado en la suma y el producto, habría producido la agrupación contraria.

3.5. Programa principal

Los ajustes en `main.cpp` fueron menores pero necesarios para poder trabajar con el conjunto completo de pruebas:

- El árbol se escribía siempre en un archivo llamado `ast.dot`. Al ejecutar las diez entradas de forma consecutiva, cada una sobrescribía a la anterior y solo quedaba la última. Ahora cada entrada genera su propio `inputN_ast.dot`.
- Se libera el árbol con `delete ast` antes de terminar.
- El programa devuelve el código de salida 1 cuando se produce un error, lo que permite que los scripts lo detecten.

4. Validación

4.1. Ejecución del conjunto de pruebas

El script `run_all_inputs.py` compila el proyecto, ejecuta las diez entradas y genera las imágenes en PNG cuando *graphviz* está disponible. En su versión original capturaba la salida del programa sin llegar a mostrarla, por lo que un error pasaba inadvertido. Ahora la imprime y lleva la cuenta de cuántas entradas se procesaron correctamente.

4.2. Casos de error

El script `test_parser.py` complementa lo anterior. Además de las diez entradas del enunciado, comprueba cinco expresiones válidas adicionales y seis inválidas, verificando en cada caso que el error producido sea del tipo esperado.

Entrada	Tipo de error	Motivo
<code>2 + \$</code>	Léxico	Carácter ajeno al lenguaje
<code>(2 + 3</code>	Sintáctico	Paréntesis sin cerrar
<code>max(2, 3))</code>	Sintáctico	Token sobrante al final
<code>2 +</code>	Sintáctico	Expresión incompleta
<code>sqrt(2, 3)</code>	Semántico	<code>sqrt</code> admite un argumento
<code>max(2)</code>	Semántico	<code>max</code> exige dos o más

Cuadro 2: Casos inválidos y error esperado en cada uno.

Estos casos fueron los más provechosos de todo el trabajo. Conseguir que un parser acepte lo correcto es relativamente sencillo; lo difícil es que rechace lo incorrecto en lugar de construir un árbol equivocado sin avisar. De hecho, el fallo del paréntesis sin cerrar solo salió a la luz al escribir esta prueba.

4.3. Resultados

Las diez entradas del enunciado se analizan correctamente y el proyecto compila sin advertencias con las opciones `-Wall -Werror`. Los veintiún casos de `test_parser.py`, entre válidos e inválidos, se superan en su totalidad.

5. Conclusiones

1. Modelar los argumentos de una función como una lista de expresiones permitió cubrir con un solo tipo de nodo tanto las funciones de un argumento como las de varios. La alternativa, una clase por función, habría supuesto más código para el mismo comportamiento.
2. El orden de trabajo no fue arbitrario. Cada módulo depende de que el anterior esté resuelto, según se resume en la tabla 3. Intentar escribir el parser antes de decidir la forma del árbol nos habría obligado a rehacerlo.
3. Conviene distinguir los tres niveles de error. Un carácter desconocido es un problema léxico, un paréntesis sin cerrar es sintáctico y una función con un número equivocado de argumentos es semántico, dado que la gramática por sí misma no puede detectarlo.
4. Ignorar el valor devuelto por `match()` hacía que ciertas entradas mal formadas se aceptaran en silencio. Comprobar siempre ese resultado convirtió un fallo silencioso en un mensaje de error concreto.

Archivo	Pregunta que resuelve	Cambio principal
token.h/.cpp	Qué símbolos existen	COMMA y FUNC
scanner.cpp	Cómo se reconocen	Decimales, coma, funciones
ast.h/.cpp	Qué árbol se construye	FuncExp con lista
parser.h/.cpp	Con qué reglas	ARGS, aridad, errores
main.cpp	Dónde se guarda	Un .dot por entrada
Scripts	Cómo se comprueba	10 entradas y 11 casos extra

Cuadro 3: Resumen del orden de trabajo.

A. Compilación y ejecución

Para procesar las diez entradas de una sola vez:

```
python3 run_all_inputs.py
```

Para compilar y analizar un archivo concreto:

```
g++ -std=c++17 -o a.out main.cpp scanner.cpp token.cpp parser.cpp ast.cpp
./a.out inputs/input5.txt
```

Cada ejecución deja dos archivos junto a la entrada: `inputN_tokens.txt` con la lista de tokens e `inputN_ast.dot` con el árbol. Este último se convierte en imagen mediante *graphviz*:

```
dot -Tpng inputs/input5_ast.dot -o inputs/input5_ast.png
```

Para ejecutar el conjunto de pruebas:

```
python3 test_parser.py
```